

COMP1531: Software Engineering Fundamentals

L. Cheung

June 9, 2026

Contents

1	Introduction	2
1.1	Course Structure	2
1.2	Introduction to Software Engineering	2
1.3	Common Software Development Models	3
1.4	JavaScript	3
1.4.1	Data Structures	3
2	Git	5
3	Unigotchi - Iteration 0	6
4	Importing Libraries	7

Chapter 1

Introduction

1.1 Course Structure

- Lab tasks are worth 10% in total
- Group project worth 45% of grade
- The final exam is worth 45%

1.2 Introduction to Software Engineering

What is the difference between Computer Science and Software Engineering.

- Computer science focuses on understanding how computers work and function
- Software engineering is about creating software for people to use, and building software that is maintainable over time

The Software Development Life Cycle is a structured approach to software development that includes:

- Requirements analysis
 - Talk to potential clients
 - Work out what is needed of the software
- Design
 - Designing an outline for the software
- Development
 - Writing the actual code to handle the task
- Testing
 - Test the program to see if it meets the initial requirements
- Deployment
 - Deploy the software to users
- Maintenance
 - Continue to develop, fixing bugs and implementing new features

Software quality and productivity is supported by three key pillars: people, technology, and processes.

1.3 Common Software Development Models

- Waterfall - linear and structured
- Prototyping - get early feedback
- Iterative development - Continuous improvement and feature addition
- Agile - Iterative development but getting feedback on each feature after it is completed

1.4 JavaScript

- Developed in 1995 so Netscape could add scripting to their browser
- Extensively used by large tech companies
- Front-end software
- NodeJS compiles and runs JavaScript code
- NodeJS is interpreted, ie. a high level language. The code gets interpreted before it gets compiled into machine language.
- JS type shenanigans
- `let` creates a variable that can be reassigned
- `const` creates a variable that cannot be reassigned. However, objects can still be manipulated.
- `${variable}` can be used to insert the value of a variable into a string
- Strings
 - String concatenation can be performed using `+`, eg. `"some text" + variable`
 - Adding variables into strings can be done using string interpolation, eg. ``Hello, my name is ${name}`` (where `name` is a variable)
- Control structures
 - Very similar syntax to C, nothing important to note

1.4.1 Data Structures

- Data structures are designed to scale complex tasks
- Two important data structures that are both collections of data are
 - Sequential collections - indexed things such as arrays
 - Associative collections - values are referenced by a string key that maps it to a value, in JS is an object

```
let student = {};  
  
student.name = "Jake";           // or student["name"]  
student.age = 25;  
student.country = "Australia";  
  
console.log(student);  
  
console.log(Object.keys(student));  
// OUTPUT: [ "name", "age", "country" ]
```

```
console.log(Object.values(student));  
// OUTPUT: [ "Jake", 25, "Australia" ]
```

```
console.log(Object.entries(student));  
// OUTPUT: [  
//   ["name", "Jake"],  
//   ["age", 25],  
//   ["country", "Australia"]  
// ]
```

- Arrays and objects have methods
- Objects are the only non-primitive type

Chapter 2

Git

To effectively work on large projects with multiple people, we need advanced methods of managing the code-base. Simple file sharing is insufficient.

Git is a program that allows distributed version control where each user has a full backup. Work can also be stored in the cloud. GitHub, GitLab, and BitBucket are software tools that implement git via a web app (COMP1531 uses GitLab).

Git can be used as

1. Version control on a single machine
2. Version control across multiple of your machines
3. Version control across a team of engineers

Files are added to a staging area with `git add .`, where `.` refers to all changes. The staged changes can be committed to the working tree with `git commit -m "message"`. Finally, to export the changes into the cloud, `git push` can be used.

`git diff` shows the difference between the working directory and staged directory.

Chapter 3

Unigotchi - Iteration 0

- Understand the requirements
- Design the function interfaces
- Set up how the team works together
- Write stub functions only (no mechanics)

1. Requirements
2. Design
3. Stub
4. MR
5. Review
6. Merge

Chapter 4

Importing Libraries

Libraries allow us to use code that other people have written in our own projects. In JavaScript, we can do so by adding the header `import fs from 'fs'`. Unlike C, the functions that are important can not be directly called. Instead, the functions exist as methods on objects, where the object name is specified when the library is imported.

We can also import our own code in other files. The functions that we want to use elsewhere can be exported by adding `export` or `export default`. The `default` keyword means that when the function is imported it will run the defaulted command. This is useful when providing an alias for the function.